



Синхронизация процессов

Операционные системы



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Что такое потоки в C#

Поток — это некая независимая последовательность инструкций для выполнения того или иного действия в программе. В одном конкретном потоке выполняется одна конкретная последовательность действий.

Совокупность таких потоков, выполняемых в программе параллельно называется многопоточностью программы.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Что такое потоки в C#

Стандартно в проектах Visual Studio существует только один основной поток — в методе Main. Всё, что в нём выполняется — выполняется последовательно строка за строкой. Но при необходимости можно «распараллелить» выполняемые процессы при помощи потоков.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Что такое потоки в C#

Большое количество потоков, с большой вероятностью, будут мешать друг другу выполнять свою работу

Поэтому потоки надо распределять исключительно в случаях, когда это действительно необходимо для ускорения работы программы либо для повышения производительности.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Что такое потоки в C#

Язык C# имеет встроенную поддержку многопоточности, а среда .NET Framework предоставляет сразу несколько классов для работы с потоками, что в купе очень помогает гибко и правильно реализовывать и настраивать многопоточность в проектах.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Что такое потоки в C#

В среде .NET Framework существует два типа потоков: основной и фоновый (вспомогательный).

В целом отличие между ними одно — если первым завершится основной поток, то фоновые потоки в его процессе будут также принудительно остановлены, если же первым завершится фоновый поток, то это не повлияет на остановку основного потока — тот будет продолжать функционировать до тех пор, пока не выполнит всю работу и самостоятельно не остановится.

Обычно при создании потока ему по умолчанию присваивается основной тип



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Для работы с потоками в C# необходимо подключить специальную директиву:

```
using System.Threading;
```

Именно она позволяет реализовывать необходимые потоки и их настройку.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

В пространстве имен System.Threading определен класс, представляющий отдельный поток - класс Thread.

Класс Thread определяет ряд методов и свойств, которые позволяют управлять потоком и получать информацию о нем. Основные свойства класса:

- **ExecutionContext:** позволяет получить контекст, в котором выполняется поток
- **IsAlive:** указывает, работает ли поток в текущий момент
- **IsBackground:** указывает, является ли поток фоновым
- **Name:** содержит имя потока



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

- **ManagedThreadId:** возвращает числовой идентификатор текущего потока
- **Priority:** хранит приоритет потока - значение перечисления **ThreadPriority**:
 - **Lowest**
 - **BelowNormal**
 - **Normal**
 - **AboveNormal**
 - **Highest**



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

По умолчанию потоку задается значение Normal. Однако можно изменить приоритет в процессе работы программы. Например, повысить важность потока, установив приоритет Highest. Среда CLR (Common language runtime — общеязыковая исполняющая среда) будет считывать и анализировать значения приоритета и на их основании выделять данному потоку то или иное количество времени.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

- ThreadState возвращает состояние потока - одно из значений перечисления ThreadState:
 - Aborted: поток остановлен, но пока еще окончательно не завершен
 - AbortRequested: для потока вызван метод Abort, но остановка потока еще не произошла
 - Background: поток выполняется в фоновом режиме
 - Running: поток запущен и работает (не приостановлен)
 - Stopped: поток завершен
 - StopRequested: поток получил запрос на остановку
 - Suspended: поток приостановлен



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

- ThreadState возвращает состояние потока - одно из значений перечисления ThreadState:
 - SuspendRequested: поток получил запрос на приостановку
 - Unstarted: поток еще не был запущен
 - WaitSleepJoin: поток заблокирован в результате действия методов Sleep или Join



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

В процессе работы потока его статус многократно может измениться под действием методов. Так, в самом начале еще до применения метода Start его статус имеет значение Unstarted. При запуске потока изменяется его статус на Running. Вызвав метод Sleep, статус изменится на WaitSleepJoin.

Кроме того статическое свойство CurrentThread класса Thread позволяет получить текущий поток

В программе на C# есть как минимум один поток - главный поток, в котором выполняется метод Main.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

```
using System.Threading;
// получаем текущий поток
Thread currentThread = Thread.CurrentThread;
//получаем имя потока
Console.WriteLine($"Имя потока: {currentThread.Name}");
currentThread.Name = "Метод Main";
Console.WriteLine($"Имя потока: {currentThread.Name}");
Console.WriteLine($"Запущен ли поток: {currentThread.IsAlive}");
Console.WriteLine($"Id потока: {currentThread.ManagedThreadId}");
Console.WriteLine($"Приоритет потока: {currentThread.Priority}");
Console.WriteLine($"Статус потока: {currentThread.ThreadState}");
```




ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Имя потока:

Имя потока: Метод Main

Запущен ли поток: True

Id потока: 1

Приоритет потока: Normal

Статус потока: Running

Так как по умолчанию свойство Name у объектов Thread не установлено, то в первом случае в качестве значения этого свойства пуста строка.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Также класс Thread определяет ряд методов для управления потоком. Основные из них:

- Статический метод `GetDomain` возвращает ссылку на домен приложения
- Статический метод `GetDomainID` возвращает id домена приложения, в котором выполняется текущий поток
- Статический метод `Sleep` останавливает поток на определенное количество миллисекунд



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Также класс Thread определяет ряд методов для управления потоком. Основные из них:

- Метод Interrupt прерывает поток, который находится в состоянии WaitSleepJoin
- Метод Join блокирует выполнение вызвавшего его потока до тех пор, пока не завершится поток, для которого был вызван данный метод
- Метод Start запускает поток



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Например, применим метод Sleep для задания задержки выполнения приложения:

```
using System.Threading;

for(int i = 0; i < 10; i++)
{
    Thread.Sleep(500);    // задержка выполнения на 500 миллисекунд
    Console.WriteLine(i);
}
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Любой поток в C# должен обязательно происходить в каком-либо методе или функции, поэтому для работы с новым потоком необходимо сначала создать для него, например, метод, который и будет точкой входа для этого потока.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Для примера создадим пустой метод под именем fornewthread, который ничего не возвращает:

```
static void fornewthread()
{
    //в действительности здесь должны находиться
инструкции, которые будут выполняться в новом
потоке
}
```




ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как создавать потоки в C#

Теперь можно создать сам поток (например, в главном методе Main) с названием mythread:

```
Thread mythread = new Thread(fornewthread);
```

Таким образом, для создания потока нужно вызвать делегат Thread, а также передать конструктору адрес метода (в скобках).



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как запускать потоки в C#

Потоки в C# начинают выполняться не сразу после их инициализации. Каждый из созданных потоков необходимо сначала запустить. Делается это следующим образом:
`имя_потока.Start();`

В случае из примера строка запуска будет такой:

```
mythread.Start();
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как запускать потоки в C#

Данный код, также как и предыдущий, записывается в методе Main сразу после инициализации самого потока. Теперь, при запуске программы, как только в Main выполнится метод Start(), начнёт работать вновь созданный поток.

После того, как выполнился Start(), работа программы «распараллеливается»: один поток начинает выполнять код из метода fornewthread, а второй поток продолжает выполнять операции, которые остались ещё не выполненными в методе Main (после Start()).



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как запускать потоки в C#

Естественно, это всё произойдёт, если в `fornewthread` или в `Main` имеется дополнительный код.

Если же один из потоков выполнит работу раньше другого, то первый будет ожидать окончания выполнения работы последнего.

А делать это он будет по причине, описанной выше: так как оба потока являются основными, то завершение какого-либо одного потока не влияет на завершение другого.

Если бы один из потоков был фоновым и «задержался» бы с работой, то при окончании работы основного потока, принудительно завершился бы и он.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

Для примера создадим проект C# с тремя потоками, каждый из которых будет выводить числа от 0 до 9.

```
static void mythread1()
{
    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine("Поток 1 выводит " + i);
    }
}
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

```
static void mythread2()  
{  
    for (int i = 0; i < 10; i++)  
    {  
        Console.WriteLine("Поток 2 выводит " + i);  
    }  
}
```




ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

```
static void Main(string[] args)
{
    Thread thread1 = new Thread(mythread1);
    Thread thread2 = new Thread(mythread2);

    thread1.Start();
    thread2.Start();

    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine("Поток 3 выводит " + i);
    }
    Console.ReadLine();
}
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

Чтобы инициализировать потоки в C#, создаются два дополнительных метода `mythread1` и `mythread2`, в которых будут выполняться соответственно потоки `thread1` и `thread2`.

Третий поток будет работать непосредственно в методе `Main`.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

После инициализации потоков их нужно запустить (третий поток заработал с самого запуска программы, его не надо инициализировать и начинать, естественно). В каждой функции происходит одно и то же действие — вывод в консоль цифр от 0 до 9, и при этом каждый поток получил от нас нумерацию от 1 до 3 для удобства восприятия.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

Результат работы программы

```
Поток 1 выводит 0
Поток 1 выводит 1
Поток 1 выводит 2
Поток 1 выводит 3
Поток 1 выводит 4
Поток 1 выводит 5
Поток 3 выводит 0
Поток 3 выводит 1
Поток 3 выводит 2
Поток 3 выводит 3
Поток 3 выводит 4
Поток 3 выводит 5
Поток 3 выводит 6
Поток 3 выводит 7
Поток 3 выводит 8
Поток 3 выводит 9
Поток 2 выводит 0
Поток 2 выводит 1
Поток 2 выводит 2
Поток 2 выводит 3
Поток 2 выводит 4
Поток 2 выводит 5
Поток 2 выводит 6
Поток 2 выводит 7
Поток 2 выводит 8
Поток 2 выводит 9
Поток 1 выводит 6
Поток 1 выводит 7
Поток 1 выводит 8
Поток 1 выводит 9
```

```
Поток 3 выводит 0
Поток 3 выводит 1
Поток 3 выводит 2
Поток 3 выводит 3
Поток 3 выводит 4
Поток 3 выводит 5
Поток 1 выводит 0
Поток 2 выводит 0
Поток 2 выводит 1
Поток 2 выводит 2
Поток 2 выводит 3
Поток 2 выводит 4
Поток 2 выводит 5
Поток 2 выводит 6
Поток 2 выводит 7
Поток 2 выводит 8
Поток 2 выводит 9
Поток 3 выводит 6
Поток 3 выводит 7
Поток 3 выводит 8
Поток 3 выводит 9
Поток 1 выводит 1
Поток 1 выводит 2
Поток 1 выводит 3
Поток 1 выводит 4
Поток 1 выводит 5
Поток 1 выводит 6
Поток 1 выводит 7
Поток 1 выводит 8
Поток 1 выводит 9
```

```
Поток 3 выводит 0
Поток 3 выводит 1
Поток 3 выводит 2
Поток 3 выводит 3
Поток 3 выводит 4
Поток 3 выводит 5
Поток 3 выводит 6
Поток 3 выводит 7
Поток 3 выводит 8
Поток 3 выводит 9
Поток 1 выводит 0
Поток 1 выводит 1
Поток 1 выводит 2
Поток 1 выводит 3
Поток 1 выводит 4
Поток 1 выводит 5
Поток 1 выводит 6
Поток 1 выводит 7
Поток 1 выводит 8
Поток 2 выводит 0
Поток 2 выводит 1
Поток 2 выводит 2
Поток 2 выводит 3
Поток 2 выводит 4
Поток 2 выводит 5
Поток 2 выводит 6
Поток 2 выводит 7
Поток 2 выводит 8
Поток 2 выводит 9
Поток 1 выводит 6
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

Пример программы с тремя потоками, каждый из которых будет выводить в консоль цифры от 0 до 9, от 10 до 19 и от 20 до 29 соответственно. Задача - вывести в консоль все эти числа последовательно от 0 до 29.

Если пренебречь приоритетами при постановке данной задачи, то никак не получится вывести все числа по очереди. Так как у всех трёх потоков одинаковый приоритет, процессору, по сути, будет всё равно, какой за каким потоки выводить, и будет выходить плохо отсортированный набор чисел.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

```
Поток 1 выводит 0
Поток 1 выводит 1
Поток 1 выводит 2
Поток 1 выводит 3
Поток 1 выводит 4
Поток 1 выводит 5
Поток 2 выводит 10
Поток 2 выводит 11
Поток 2 выводит 12
Поток 2 выводит 13
Поток 2 выводит 14
Поток 2 выводит 15
Поток 2 выводит 16
Поток 2 выводит 17
Поток 2 выводит 18
Поток 2 выводит 19
Поток 1 выводит 6
Поток 1 выводит 7
Поток 1 выводит 8
Поток 1 выводит 9
Поток 3 выводит 20
Поток 3 выводит 21
Поток 3 выводит 22
Поток 3 выводит 23
Поток 3 выводит 24
Поток 3 выводит 25
Поток 3 выводит 26
Поток 3 выводит 27
Поток 3 выводит 28
Поток 3 выводит 29
```




ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

```
static void mythread1()
{
    for (int i = 0; i < 10; i++)
    {
        Console.WriteLine("Поток 1 выводит " + i);
    }
}
static void mythread2()
{
    for (int i = 10; i < 20; i++)
    {
        Console.WriteLine("Поток 2 выводит " + i);
    }
}
```



Реализация потоков в C#

Как работают потоки в C#

```
static void mythread3()
{
    for (int i = 20; i < 30; i++)
    {
        Console.WriteLine("Поток 3 выводит " + i);
    }
}
static void Main(string[] args)
{
    Thread thread1 = new Thread(mythread1);
    Thread thread2 = new Thread(mythread2);
    Thread thread3 = new Thread(mythread3);
    thread1.Priority = ThreadPriority.Highest;
    thread2.Priority = ThreadPriority.AboveNormal;
    thread3.Priority = ThreadPriority.Lowest;
```



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

```
thread1.Start();  
thread2.Start();  
thread3.Start();  
  
Console.ReadLine();  
}  
}
```

В методе Main заданы приоритеты потокам. Первому потоку установлен самый высокий приоритет. Второй поток имеет приоритет чуть ниже, поэтому будет выполняться вторым, третий поток имеет самый низкий приоритет, поэтому будет выполняться после всех остальных потоков.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Реализация потоков в C#

Как работают потоки в C#

Результат работы программы

```
Поток 3 выводит 0
Поток 3 выводит 1
Поток 3 выводит 2
Поток 3 выводит 3
Поток 3 выводит 4
Поток 3 выводит 5
Поток 3 выводит 6
Поток 3 выводит 7
Поток 3 выводит 8
Поток 3 выводит 9
Поток 2 выводит 10
Поток 2 выводит 11
Поток 2 выводит 12
Поток 2 выводит 13
Поток 2 выводит 14
Поток 2 выводит 15
Поток 2 выводит 16
Поток 2 выводит 17
Поток 2 выводит 18
Поток 2 выводит 19
Поток 1 выводит 20
Поток 1 выводит 21
Поток 1 выводит 22
Поток 1 выводит 23
Поток 1 выводит 24
Поток 1 выводит 25
Поток 1 выводит 26
Поток 1 выводит 27
Поток 1 выводит 28
Поток 1 выводит 29
```


**При переходе от последовательных
решений к параллельным возникает
проблема синхронизации**

- ✓ обеспечение согласованных действий параллельно работающих модулей при выполнении этапов алгоритма
- ✓ обеспечение целостности используемых данных
- ✓ извещение модулями друг друга о произошедших событиях
- ✓ и т.д.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Потребность в синхронизации потоков возникает только в мультипрограммной операционной системе и связана с совместным использованием аппаратных и информационных ресурсов вычислительной системы.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Синхронизация необходима для исключения гонок и тупиков

- при обмене данными между потоками;
- при разделении данных;
- при доступе к процессору;
- при доступе устройствам ввода-вывода.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Потоки в общем случае протекают независимо, асинхронно друг другу.

Это справедливо по отношению:

- ✓ к потокам одного процесса, выполняющим общий программный код;
- ✓ к потокам разных процессов, каждый из которых выполняет собственную программу.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Любое взаимодействие процессов или потоков связано с их синхронизацией, которая заключается в согласовании их скоростей путем приостановки потока до наступления некоторого события и последующей его активизации при наступлении этого события.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Синхронизация лежит в основе любого взаимодействия потоков, связано ли это взаимодействие с разделением ресурсов или с обменом данными.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Процессы взаимодействуют для выполнения общей задачи, для этого процессу требуется:

- возможность запрашивать сервисы у другого процесса;
- ждать выполнения соответствующих операций.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Процессы соревнуются за монопольное использование сервисов или ресурсов. Соревнующимся процессам нужно:

- ждать получения общего ресурса;
- сигнализировать о завершении его использования.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Для успешного взаимодействия процессов необходимы два механизма.

- ✓ *Условная синхронизация.* Процессам, выполняющим общую задачу, необходимо синхронизироваться, и для этого им нужно ЖДАТЬ друг друга и СИГНАЛИЗИРОВАТЬ друг другу.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

✓ *Взаимное исключение.*

Соревнующиеся процессы должны ЖДАТЬ освобождения общего ресурса и СИГНАЛИЗИРОВАТЬ об окончании его использования.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Цели и средства синхронизации

Пренебрежение вопросами синхронизации процессов, выполняющихся в режиме мультипрограммирования, может привести к их неправильной работе или даже к краху системы.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Критическая секция



ВЯТСКОЕ
ГОСУДАРСТВЕННОЕ
УНИВЕРСИТЕТ

Процесс
P1

Проблема

Процесс
P2

p1: ...x:=x-1;...

p2: ...x:=x-1;...



Рассмотрим два процесса p1 и p2 уменьшающих асинхронно значение общей переменной x, представляющей число единиц ресурса.

**Критическая
секция**

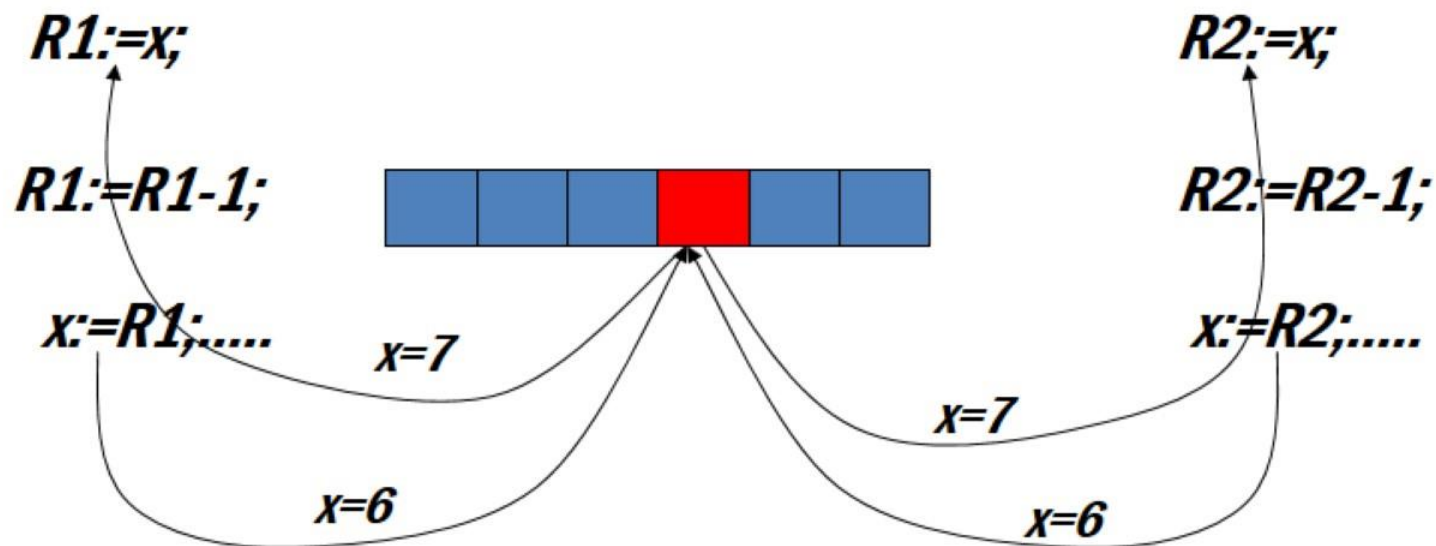


ВЯТСКОЕ
ГОСУДАРСТВЕННОЕ
УНИВЕРСИТЕТ

Процесс
P1

Проблема

Процесс
P2



**Критическая
секция**



ВЯТСКОЕ
ГОСУДАРСТВЕННОЕ
УНИВЕРСИТЕТ

Процесс
P1

Проблема

Процесс
P2

*Критическая
секция процесса P1*

$R1:=x;$
 $R1:=R1-1;$
 $x:=R1;.....$

$R2:=x;$
 $R2:=R2-1;$
 $x:=R2;.....$

*Критическая секция
процесса P2*



Критические данные

**Критическая
секция**



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Проблема

Для предотвращения проблем, возникающих, когда два или более процессов используют общую структуру данных, необходим специальный механизм,

- управляющий доступом к этой структуре и
- не позволяющий нескольким процессам обращаться к ней либо изменять ее одновременно



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Определение

Критическая секция – это часть программы, в которой осуществляется доступ к разделяемым данным



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Понятие

Чтобы исключить эффект гонок по отношению к некоторому ресурсу, необходимо обеспечить, чтобы в каждый момент в критической секции, связанной с этим ресурсом, находился максимум один процесс.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Понятие

Существует ряд механизмов, гарантирующих, что в каждый конкретный момент времени критическая секция, связанная с определенным элементом данных, будет выполняться только одним процессом.

Эти механизмы обеспечивают взаимное исключение монопольный доступ к общим данным.



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Проектирование параллельных вычислительных процессов



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Задача взаимного исключения. Постановка задачи

Необходимо согласовать работу $n > 1$ параллельных потоков при использовании некоторого критического ресурса таким образом, чтобы удовлетворить следующим требованиям:

- ✓ взаимногоисключения должны выполняться принудительно: одновременно внутри критической секции должно находиться не более одного потока
- ✓ остановка какого-либо потока вне его критической секции не должна влиять на дальнейшую работу потоков по использованию критического ресурса



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Требования для согласования работы $n > 1$ параллельных потоков при использовании некоторого критического ресурса

- ✓ критические секции не должны иметь приоритета в отношении друг друга
- ✓ решение о вхождении потоков в их критические секции при одинаковом времени поступления запросов на такое вхождение и равноприоритетности потоков не откладывается на неопределенный срок, а является конечным во времени

*Проектирование параллельных
вычислительных процессов*



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Требования для согласования работы $n > 1$ параллельных потоков при использовании некоторого критического ресурса

- ✓ относительные скорости развития потоков неизвестны и произвольны
- ✓ любой поток может переходить в любое состояние, отличное от активного, вне пределов своей критической секции



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Требования для согласования работы $n > 1$ параллельных потоков при использовании некоторого критического ресурса

- ✓ освобождение критического ресурса и выход из критической секции должны быть произведены потоком, использующим критический ресурс, за конечное время

Реализация критических секций с использованием блокирующих переменных

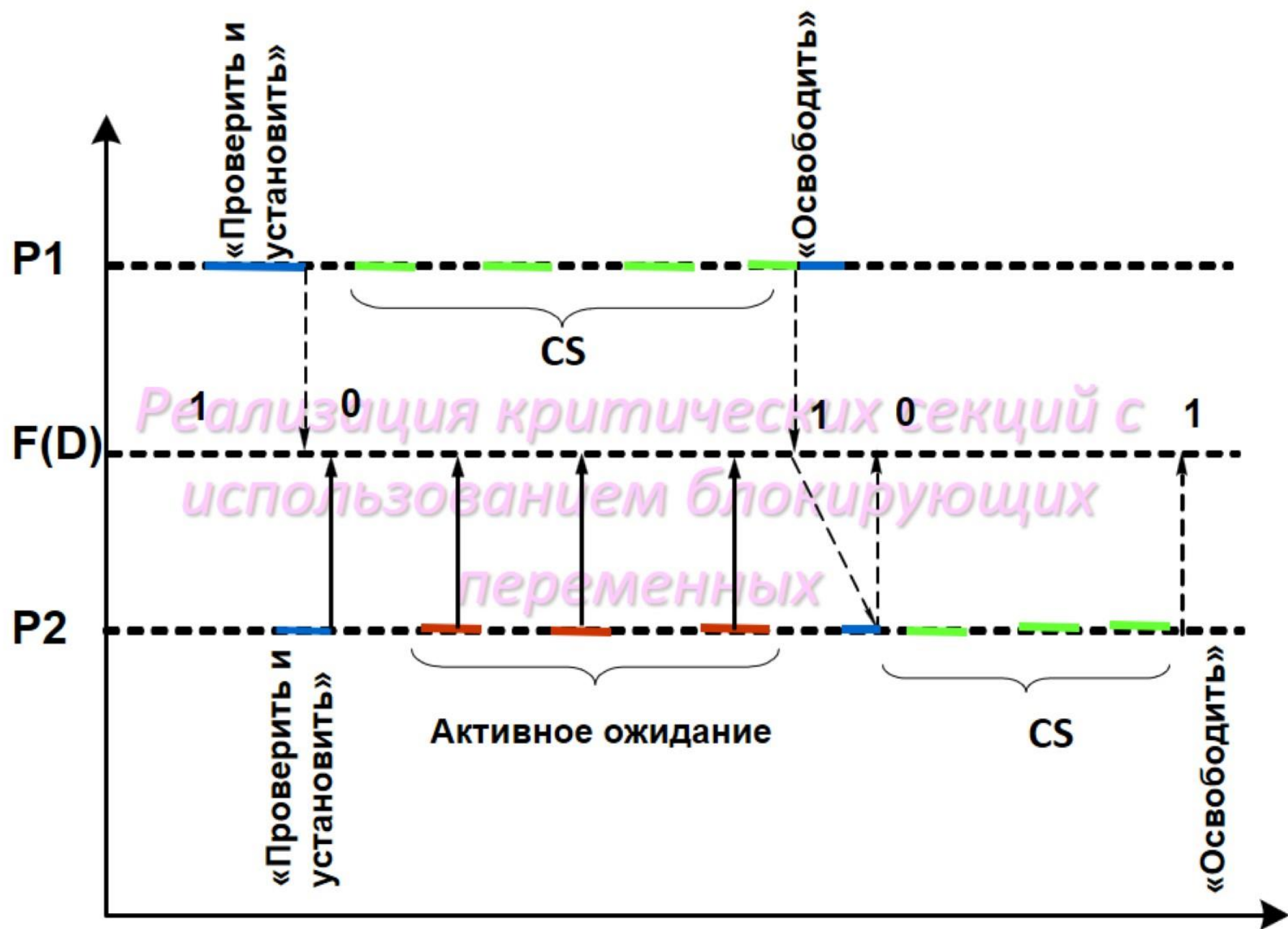
К разделяемому ресурсу D



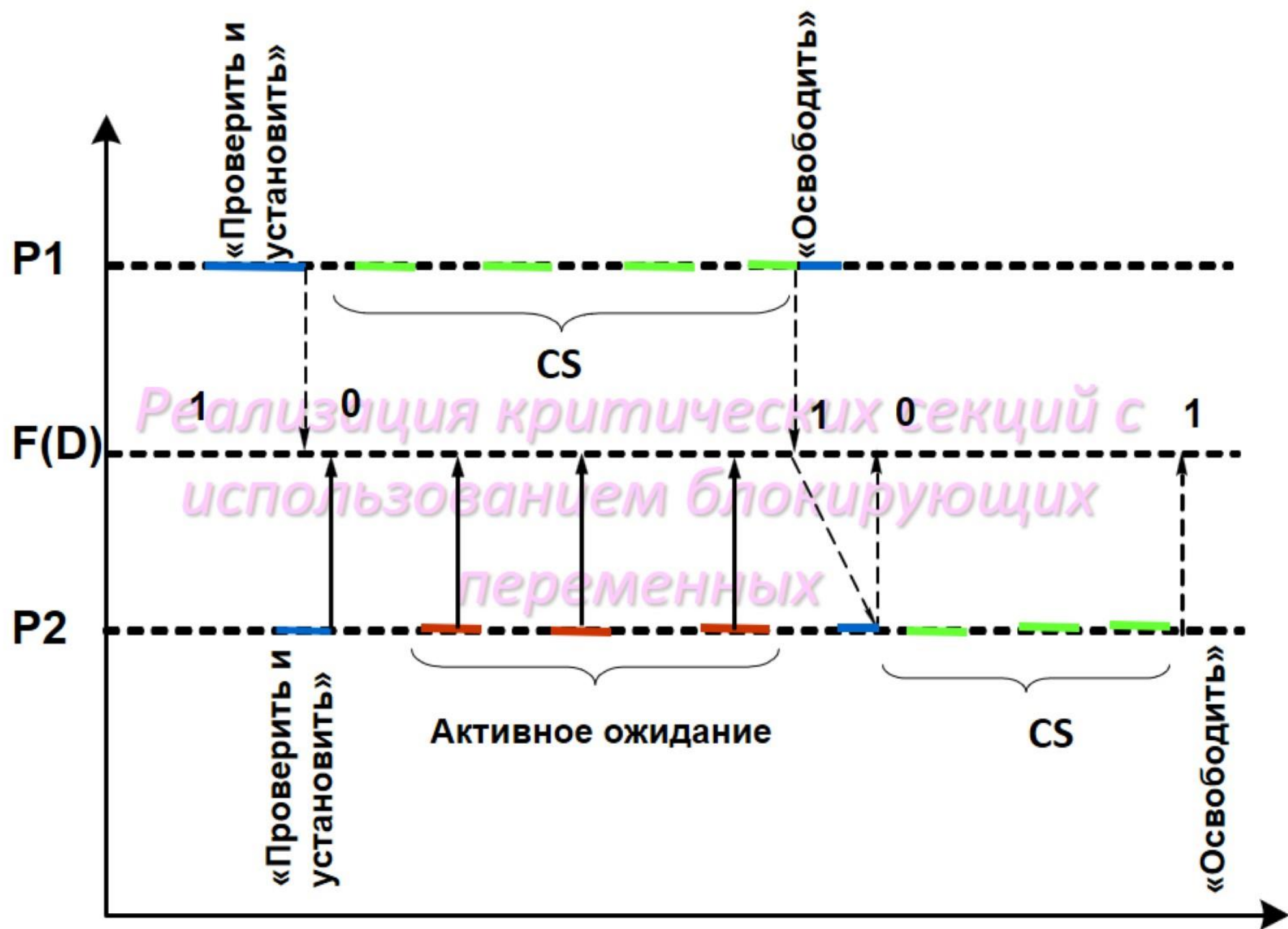
Неделимая операция
«проверка-установка»

При работе с **признаком занятости (блокировки)** ресурса поток использует "активное ожидания", то есть крутится в цикле, ожидая освобождения признака блокировки

Проектирование параллельных вычислительных процессов



Проектирование параллельных вычислительных процессов



Проектирование параллельных вычислительных процессов



ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

Использование блокирующих переменных

Недостатки активного ожидания

- ✓ Активное ожидание работает, но оно очень затратно: если какой-то поток находится в состоянии активного ожидания, поток, захвативший признак блокировки, не может продолжать свое исполнение и никакой другой поток тоже не может
- ✓ Активное ожидание используется как база для построения высокоуровневых механизмов
- ✓ Существуют ситуации, когда активное ожидание оправдано





ВЯТСКИЙ
ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ

- <https://learn.microsoft.com/ru-ru/dotnet/standard/threading/overview-of-synchronization-primitives>
- <https://vscode.ru/prog-lessons/potoki-v-si-sharp.html>
- <https://metanit.com/sharp/tutorial/11.1.php>